

+++ date = '2026-02-13T18:16:48-08:00' draft = false title = 'Practica3: El paradigma funcional' +++

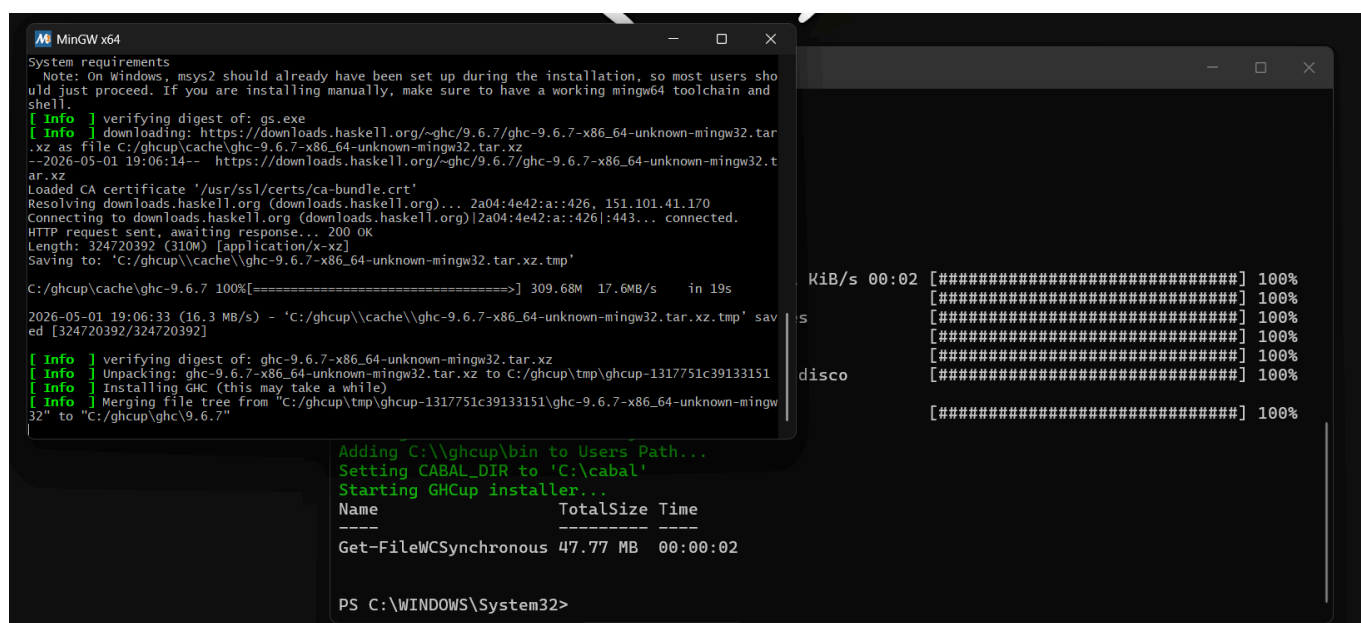
Reporte – Desarrollo de aplicación en Haskell

Primera sesión: Instalación del entorno de desarrollo

Para comenzar a trabajar con Haskell, se accedió a la página oficial de descargas:

<https://www.haskell.org/downloads/>

En esta página se indica que la instalación del entorno se realiza mediante GHCup, una herramienta que permite instalar automáticamente los componentes necesarios.



```
MinGW x64
System requirements
Note: On Windows, msys2 should already have been set up during the installation, so most users should just proceed. If you are installing manually, make sure to have a working mingw64 toolchain and shell.
[ Info ] verifying digest of: gs.exe
[ Info ] downloading: https://downloads.haskell.org/~ghc/9.6.7/ghc-9.6.7-x86_64-unknown-mingw32.tar.xz as file C:/ghcup/cache/ghc-9.6.7-x86_64-unknown-mingw32.tar.xz
--2026-05-01 19:06:14-- https://downloads.haskell.org/~ghc/9.6.7/ghc-9.6.7-x86_64-unknown-mingw32.tar.xz
Loaded CA certificate '/usr/ssl/certs/ca-bundle.crt'
Resolving downloads.haskell.org (downloads.haskell.org)... 2a04:4e42:a::426, 151.101.41.170
Connecting to downloads.haskell.org (downloads.haskell.org)|2a04:4e42:a::426|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 324720392 (310M) [application/x-xz]
Saving to: 'C:/ghcup/cache/ghc-9.6.7-x86_64-unknown-mingw32.tar.xz.tmp'

C:/ghcup/cache/ghc-9.6.7 100%[=====] 309.68M 17.6MB/s in 19s

2026-05-01 19:06:33 (16.3 MB/s) - 'C:/ghcup/cache/ghc-9.6.7-x86_64-unknown-mingw32.tar.xz.tmp' saved [324720392/324720392]

[ Info ] verifying digest of: ghc-9.6.7-x86_64-unknown-mingw32.tar.xz
[ Info ] Unpacking: ghc-9.6.7-x86_64-unknown-mingw32.tar.xz to C:/ghcup/tmp/ghcup-1317751c39133151
[ Info ] Installing GHC (this may take a while)
[ Info ] Merging file tree from "C:/ghcup/tmp/ghcup-1317751c39133151\ghc-9.6.7-x86_64-unknown-mingw32" to "C:/ghcup/ghc/9.6.7"

Adding C:\ghcup\bin to Users Path...
Setting CABAL_DIR to 'C:\cabal'
Starting GHCup installer...
Name TotalSize Time
----
Get-FileWCSynchronous 47.77 MB 00:00:02

PS C:\WINDOWS\System32>
```

Proceso de instalación

1. Se ingresó a la página de GHCup
2. Se copió el comando de instalación
3. Se ejecutó en PowerShell sin permisos de administrador
4. Se aceptaron las opciones recomendadas

Durante este proceso se instalaron los siguientes componentes:

- GHC → compilador de Haskell
- HLS → servidor del lenguaje
- Stack → manejador de paquetes
- Cabal → herramienta de construcción

Problemas durante la instalación

Durante la instalación se presentaron errores relacionados con rutas del sistema en Windows, lo cual impidió que algunas herramientas se configuraran correctamente.

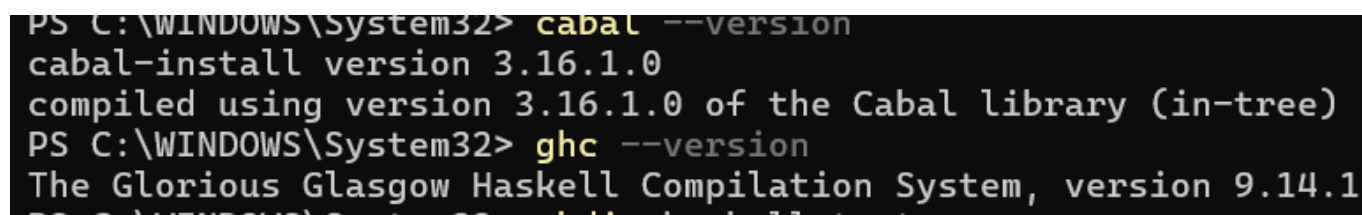
Para solucionarlo:

- Se cambió el directorio de trabajo a la carpeta del usuario
- Se ejecutaron comandos manuales con GHCup
- Se instaló Cabal manualmente

Verificación

Se comprobó la instalación con:

```
ghc --version
```



```
PS C:\WINDOWS\System32> cabal --version
cabal-install version 3.16.1.0
compiled using version 3.16.1.0 of the Cabal library (in-tree)
PS C:\WINDOWS\System32> ghc --version
The Glorious Glasgow Haskell Compilation System, version 9.14.1
```

Confirmando que el compilador se instaló correctamente.

Segunda sesión: Introducción a Haskell

Antes de comenzar la práctica, se revisó material introductorio sobre el lenguaje:

https://wiki.haskell.org/Haskell_Tutorial_for_C_Programmers

El objetivo fue comprender el paradigma funcional y la sintaxis básica de Haskell.

Funcionamiento de la aplicación TODO

La aplicación TODO desarrollada en Haskell es un programa de consola que permite gestionar una lista de tareas mediante comandos ingresados por el usuario.

El programa inicia mostrando un menú de opciones, donde el usuario puede interactuar escribiendo comandos específicos. Internamente, la aplicación utiliza una lista de tipo `String` para almacenar las tareas y una función recursiva que mantiene el programa en ejecución hasta que el usuario decide salir.

Flujo de ejecución

1. El programa inicia y muestra los comandos disponibles.
2. Se solicita al usuario que ingrese un comando.
3. Dependiendo del comando, se ejecuta una acción:
 - Agregar una tarea
 - Listar tareas
 - Eliminar una tarea
 - Salir del programa
4. La lista de tareas se actualiza dinámicamente en cada interacción.
5. El programa continúa ejecutándose hasta que se ingresa el comando `q`.

Archivos en Haskell

Los archivos de código fuente utilizan la extensión:

.hs

Proceso de desarrollo

- Se creó el proyecto: stack new todo
- Se modificaron los archivos principales: Main.hs → controla la ejecución Lib.hs → contiene la lógica
- Se compiló y ejecutó: stack run Funcionamiento de la aplicación

Comandos disponibles

- `+ tarea` → Agrega una nueva tarea a la lista
- `l` → Muestra todas las tareas registradas
- `- n` → Elimina la tarea en la posición indicada
- `q` → Finaliza la ejecución del programa

Las tareas se almacenan en una lista de tipo String.

###Ejemplo de uso

- `'+ tarea1'`
- `'+tarea2'`
- `l`
- `1.tarea1`
- `2.tarea2`
- `1`
- `l`
- `1.tarea2 Implementación`

Se utilizaron los siguientes conceptos:

- Listas
- Funciones recursivas
- Entrada de usuario (getline)
- Validación de datos
- Conclusión

Satck run para correr el haskell

```
Commands:
+ <String> - Add a TODO entry
- <Int>     - Delete the numbered entry
l           - List todo
q           - Quit
Escribe comando:
+ tarea1
Tarea agregada
Escribe comando:
+ tarea2
Tarea agregada
Escribe comando:
+ tarea3
Tarea agregada
Escribe comando:
l
1. tarea1
2. tarea2
3. tarea3
Escribe comando:
- 2
Tarea eliminada
Escribe comando:
l
1. tarea1
2. tarea3
Escribe comando:
q
PS C:\PORTAFOLIO\haskell-test\todo> |
```

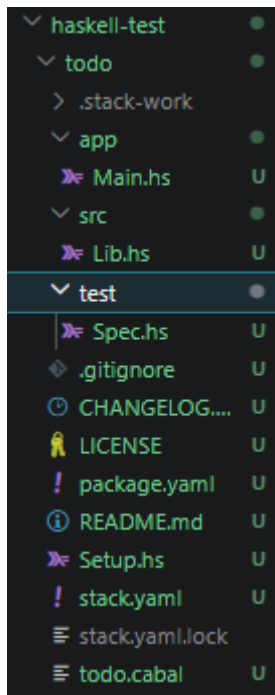
Codigo main.hs modificado

```
haskell-test > todo > app > Main.hs
1  module Main where
2
3  import Lib (prompt)
4
5  main :: IO ()
6  main = do
7      putStrLn "Commands:"
8      putStrLn "+ <String> - Add a TODO entry"
9      putStrLn "- <Int>   - Delete the numbered entry"
10     putStrLn "l         - List todo"
11     putStrLn "q         - Quit"
12     prompt []
```

Codigo libs.hs modificado

```
haskell-test > todo > src > ✖ Lib.hs
1  module Lib (prompt) where
2
3  import Text.Read (readMaybe)
4
5  prompt :: [String] -> IO ()
6  prompt todos = do
7      putStrLn "Escribe comando:"
8      command <- getLine
9      case words command of
10
11          ["l"] -> do
12              if null todos
13              then putStrLn "No hay tareas"
14              else mostrarConIndice todos
15              prompt todos
16
17          ["q"] -> return ()
18
19          ("+":tarea) -> do
20              let nuevos = todos ++ [unwords tarea]
21              putStrLn "Tarea agregada"
22              prompt nuevos
23
24          ["-", nStr] -> do
25              case readMaybe nStr of
26                  Just n ->
27                      if n > 0 && n <= length todos
28                      then do
29                          let nuevos = eliminar (n-1) todos
30                          putStrLn "Tarea eliminada"
31                          prompt nuevos
32                      else do
33                          putStrLn "Índice fuera de rango"
34                          prompt todos
35                  Nothing -> do
36                      putStrLn "Número inválido"
37                      prompt todos
38
39          _ -> do
40              putStrLn "Comando no válido"
41              prompt todos
42
43  mostrarConIndice :: [String] -> IO ()
44  mostrarConIndice xs = mapM_ putStrLn (zipWith (\i x -> show i ++ ". " ++ x) [1..] xs)
45
46  eliminar :: Int -> [a] -> [a]
47  eliminar i xs = take i xs ++ drop (i+1) xs
```

Carpeta con datos de haskell



Conclusión

Haskell utiliza el paradigma funcional, lo cual representa una forma diferente de programar en comparación con lenguajes tradicionales.

A pesar de algunos problemas durante la instalación, se logró configurar el entorno correctamente. La aplicación TODO permitió comprender cómo manejar datos, listas y entrada del usuario en Haskell.